



Introduction to Database Systems Spring 2025, Lecture 3: Normal Forms

Why Normal Forms?

Functional Dependencies

Normal Forms

- 1st Normal Form
- 2nd Normal Form
- 3rd Normal Form
- BC Normal Form

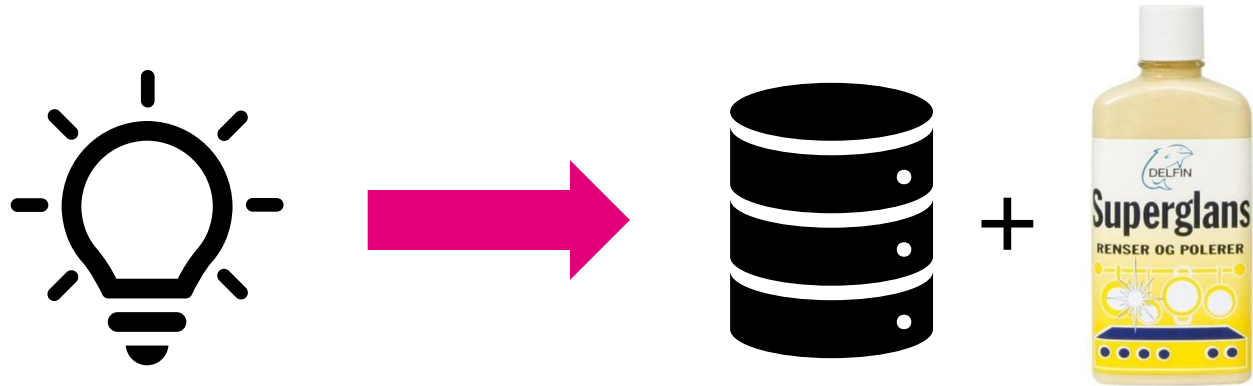
Normalization

Intended Learning Outcomes

1. Suggest a database design in the ER model and convert to a relational database schema in a suitable normal form.
2. Write SQL queries, involving multiple relations, compound conditions, grouping, aggregation, and subqueries.
3. Use relational DBMSs from a conventional programming language in a secure manner.
4. Analyze/predict/improve query processing efficiency of the designed database using indices.
5. Reflect upon the evolution of the hardware and storage hierarchy and its impact on data management system design.
6. Discuss the pros and cons of different classes of data systems for modern analytics and data science applications.

Normal Form = Clean Database Schema

1. Suggest a database design in the ER model and convert to a relational database schema in a **suitable normal form**.



Convert your idea of an app into a database schema

game

web site

business application

anything that stores data

Why?

Database Design

- How do we design a “good” database schema?
- We want to ensure the integrity of the data
- We want to get good performance

Example of a “bad” database

CoffeeShop (PurchaseDate: `date`, Cashier: `string`, ProductName: `string`,
Price: `int` , UserName: `string`, Email: `string`, Password: `string`)

PurchaseDate	Cashier	ProductName	Price	UserName	Email	Password
Feb 11, 9:55	Mads	Tea	100	Martin	mhent	***
Feb 12, 10:03	Anna	Small	85	Martin	mhent	***
Feb 12, 10:05	Anna	Small	85	Omar	omsh	***
Feb 12, 10:06	Anna	Large	100	Omar	omsh	***

Example of a “bad” database

CoffeeShop (PurchaseDate: `date`, Cashier: `string`, ProductName: `string`,
Price: `int` , UserName: `string`, Email: `string`, Password: `string`)

PurchaseDate	Cashier	ProductName	Price	UserName	Email	Password
Feb 11, 9:55	Mads	Tea	100	Martin	mhent	***
Feb 12, 10:03	Anna	Small	85	Martin	mhent	***
Feb 12, 10:05	Anna	Small	85	Omar	omsh	***
Feb 12, 10:06	Anna	Large	100	Omar	omsh	***

If there is only one cashier per shift, we are storing redundant information.

Example of a “bad” database

CoffeeShop (PurchaseDate: `date`, Cashier: `string`, ProductName: `string`,
Price: `int` , UserName: `string`, Email: `string`, Password: `string`)

PurchaseDate	Cashier	ProductName	Price	UserName	Email	Password
Feb 11, 9:55	Mads	Tea	100	Martin	mhent	***
Feb 12, 10:03	Anna	Small	85	Martin	mhent	***
Feb 12, 10:05	Anna	Small	85	Omar	omsh	***
Feb 12, 10:06	Anna	Large	100	Omar	omsh	***

Small coffees always cost 85 DKK. No
need to store this multiple times.

Example of a “bad” database

CoffeeShop (PurchaseDate: `date`, Cashier: `string`, ProductName: `string`,
Price: `int` , UserName: `string`, Email: `string`, Password: `string`)

PurchaseDate	Cashier	ProductName	Price	UserName	Email	Password
Feb 11, 9:55	Mads	Tea	100	Martin	mhent	***
Feb 12, 10:03	Anna	Small	85	Martin	mhent	***
Feb 12, 10:05	Anna	Small	85	Omar	omsh	***
Feb 12, 10:06	Anna	Large	100	Omar	omsh	***

What if a user changes their email or password? We would need to update it multiple times.

Database is not normalized!

Anomalies can occur:

- Insertion anomaly
- Deletion anomaly
- Update anomaly

Insertion anomaly examples

- If inserting a new row into CoffeeShop, we always must include the correct cashier, price, email, and password.
- Inserting a new cashier who has not worked yet is not possible.
- Inserting a new user who has not made a purchase is not possible.
- Inserting a new product that has not been purchased is not possible.

Deletion anomaly example

- Deleting a product leads to deleting all its purchases and perhaps users that only drank this product.

Update anomaly example

- Updating a user email or user password necessitates multiple updates in the CoffeeShop table. May lead to inconsistencies.

“Better” database

Shift (Day: `date`, Cashier: `string`)

Day	Cashier
Feb 11	Mads
Feb 12	Anna

User (UserName: `string`, Email: `string`, Password: `string`)

UserName	Email	Password
Martin	mhent	***
Omar	omsh	***

Product (ProductName: `string`, Price: `int`)

ProductName	Price
Tea	100
Small	85
Large	100

Purchase (ProductDate: `date`, ProductName: `string`, UserName: `string`)

PurchaseDate	ProductName	UserName
Feb 11, 9:55	Tea	Martin
Feb 12, 10:03	Small	Martin
Feb 12, 10:05	Small	Omar
Feb 12, 10:06	Large	Omar

Why is it better?

2-minute discussion with your neighbor

- Why is it better?
- What could be disadvantages?

Why is it better?

No more anomalies

- Inserting users, products, shifts independently
- Deleting products and users possible without impacting other data
- Updating email, password in one row only

No duplicate information

- Reduced risk of inconsistencies
- Efficient use of storage space

→ “Bad” = not normalized*

“Better” = normalized relational model



Why Normal Forms?

Functional Dependencies

Normal Forms

- 1st Normal Form
- 2nd Normal Form
- 3rd Normal Form
- BC Normal Form

Normalization

Functional Dependency Definition

A **functional dependency** $X \rightarrow Y$ between two sets of attribute types X and Y implies that a value of X uniquely determines a value of Y .

In our example:

UserName $\rightarrow$ (Email, Password)
ProductName $\rightarrow$ Price
Day $\rightarrow$ Cashier

Functional Dependency

Functional dependencies are important

- Allow to decide whether a database design is normalized

Identify a “bad” design

→ Check for normal forms using functional dependencies

Functional Dependency

Note:

You can check if an instance of a relation violates a functional dependency (FD), but you cannot prove that a functional dependency exists using an instance.

Example FD: Day $\rightarrow$ Cashier

Instance satisfies FD but
does not prove it exists

Day	Cashier
Feb 11	Mads
Feb 12	Anna

Instance violates FD

Day	Cashier
Feb 11	Mads
Feb 12	Anna
Feb 12	Mads

2-Minute Exercise

What functional dependencies exist?

- Addition: A customer can give one rating per product
- Assumption 1: Each email address is linked to a single signup
- Assumption 2: Only one cashier is assigned per day
- Assumption 3: Multiple purchases can occur within a single minute

PurchaseDate	Cashier	ProductName	Price	UserName	Email	Password	Rating
Feb 11, 9:55	Mads	Tea	100	Martin	mhent	***	2 stars
Feb 12, 10:03	Anna	Small	85	Martin	mhent	***	5 stars
Feb 12, 10:05	Anna	Small	85	Omar	omsh	***	4 stars
Feb 12, 10:06	Anna	Large	100	Omar	omsh	***	5 stars

2-Minute Exercise

What functional dependencies exist?

PurchaseDate	Cashier	ProductName	Price	UserName	Email	Password	Rating
Feb 11, 9:55	Mads	Tea	100	Martin	mhent	***	2 stars
Feb 12, 10:03	Anna	Small	85	Martin	mhent	***	5 stars
Feb 12, 10:05	Anna	Small	85	Omar	omsh	***	4 stars
Feb 12, 10:06	Anna	Large	100	Omar	omsh	***	5 stars

PurchaseDate → Cashier

ProductName → Price

UserName → (Email, Password)

Email → (UserName, Password)

(ProductName, UserName) → Rating

(ProductName, Email) → Rating

Full & Partial Functional Dependency Definition

A functional dependency $X \rightarrow Y$ is a **full functional dependency** if removal of any attribute type A from X means that the dependency does not hold anymore.

A functional dependency $X \rightarrow Y$ is a **partial functional dependency** if an attribute type A from X can be removed from X and the dependency still holds.

(ProductName, UserName) → Rating

is a **full** functional dependency. Rating depends on ProductName **and** UserName.

(ProductName, UserName) → Password

is **partial** functional dependency because Password depends on UserName only. ProductName can be removed, and the dependency still holds.

Prime/non-prime Definition

A **prime attribute type** is an attribute type that is part of a candidate key.

Attribute types that are not part of any candidate key are **non-prime**.

In our example:

User (UserName: string, Email: string, Password: string)

UserName	Email	Password
Martin	mhent	***
Omar	omsh	***

UserName and Email* are prime
Password is non-prime

*If each email address is linked to a single signup.

Why Normal Forms?

Functional Dependencies

Normal Forms

- 1st Normal Form
- 2nd Normal Form
- 3rd Normal Form
- BC Normal Form

Normalization

First Normal Form (1NF) Definition

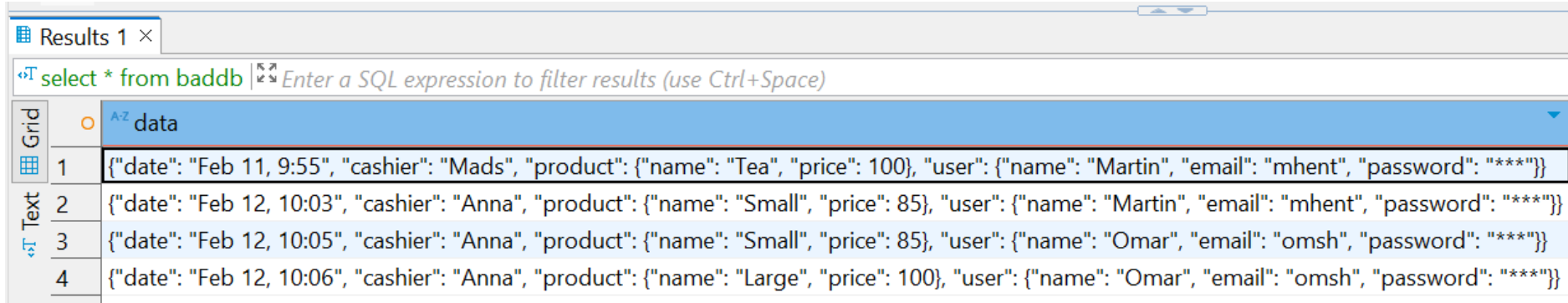
The **first normal form (1NF)** states that every attribute type of a relation must be atomic and single-valued.

- Atomic: not composed
- Single-valued: not a list or a set

Examples NOT in 1NF

NOT in first normal form because not atomic:

BadCoffeeShop (Data(PurchaseDate, Cashier, Product(Name, Price), User(Name, Email, Password)))



The screenshot shows a database query results window with the title 'Results 1'. The query entered is 'select * from baddb'. The results are displayed in a table with a 'Grid' view selected. The table has four rows of JSON data. The first row shows a purchase on Feb 11, 9:55, by Mads, for Tea, by user Martin. The next two rows show purchases on Feb 12, 10:03 and 10:05, both by Anna, for Small coffee, by user Omar. The fourth row shows a purchase on Feb 12, 10:06, by Anna, for Large coffee, by user Omar.

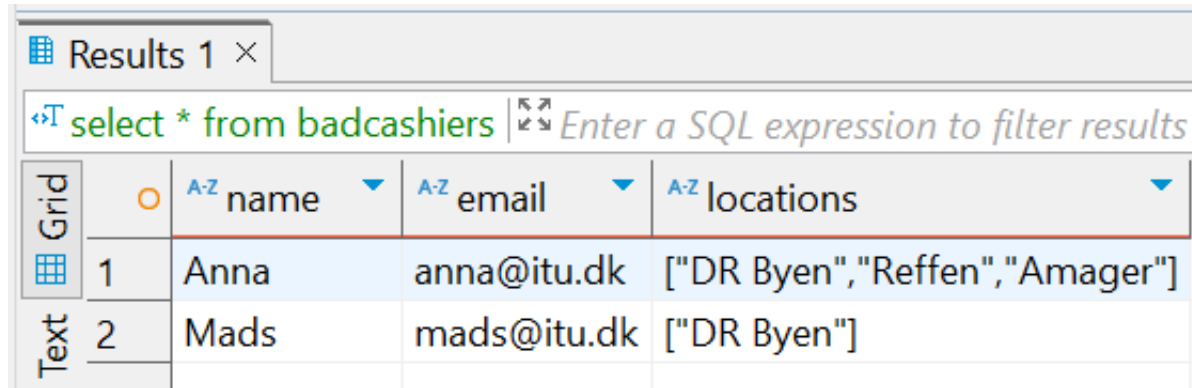
	A-Z data
1	{"date": "Feb 11, 9:55", "cashier": "Mads", "product": {"name": "Tea", "price": 100}, "user": {"name": "Martin", "email": "mhent", "password": "****"}}
2	{"date": "Feb 12, 10:03", "cashier": "Anna", "product": {"name": "Small", "price": 85}, "user": {"name": "Martin", "email": "mhent", "password": "****"}}
3	{"date": "Feb 12, 10:05", "cashier": "Anna", "product": {"name": "Small", "price": 85}, "user": {"name": "Omar", "email": "omsh", "password": "****"}}
4	{"date": "Feb 12, 10:06", "cashier": "Anna", "product": {"name": "Large", "price": 100}, "user": {"name": "Omar", "email": "omsh", "password": "****"}}

```
create table badcoffeeshop(data json);
insert into badcoffeeshop
select * from read_json_objects(
    'C:\Users\mhent\duckdb\baddb.json', format = 'unstructured');
select * from badcoffeeshop;
```

Examples NOT in 1NF

NOT in first normal form because not single-valued:

BadCashiers (Name, Email, Locations)



Results 1 ×

select * from badcashiers

	name	email	locations
1	Anna	anna@itu.dk	["DR Byen", "Reffen", "Amager"]
2	Mads	mads@itu.dk	["DR Byen"]

```
create table badcashiers(name string, email string, locations json);
insert into badcashiers values
  ('Anna', 'anna@itu.dk', json('["DR Byen", "Reffen", "Amager"]')),
  ('Mads', 'mads@itu.dk', json('["DR Byen"]'));
select * from badcashiers;
```

Bringing into 1NF

1. Decompose composite attribute types
2. Split multi-valued attribute types into relations

Bringing into 1NF

Relation with composite attribute types:

BadCoffeeShop (Data(PurchaseDate, Cashier, Product(Name, Price),
User(Name, Email, Password)))

Decomposing into a relation with atomic attribute types:

CoffeeShop (PurchaseDate, Cashier, ProductName, Price, UserName,
Email, Password))

Note: I renamed the relation and some attributes
and omitted types.

Bringing into 1NF

Relation with multi-values attribute types:

BadCashiers (Name, Email, **Locations**)

Split into two relations:

Cashiers (Name, Email)

Locations(Name, Location)

Bringing into 1NF

BadCashiers (Name, Email, Locations)

Name	Email	Locations
Anna	anna@itu.dk	DR Byen, Reffen, Amager
Mads	mads@itu.dk	DR Byen

Cashiers (Name, Email)

Name	Email
Anna	anna@itu.dk
Mads	mads@itu.dk

Locations (Name, Location)

Name	Location
Anna	DR Byen
Anna	Reffen
Anna	Amager
Mads	DR Byen

Second Normal Form (2NF) Definition

A relation R is in **second normal form (2NF)** if it satisfies 1NF and every non-prime attribute type A in R is fully functionally dependent on any key of R .

Bringing into 2NF

Decompose the relation:

- One new relation for every partial key and its dependent attribute types
- Keep a relation with the original primary key and attribute types fully functionally dependent on it

Bringing into 2NF

CoffeeShop (PurchaseDate, Cashier, ProductName, Price, UserName, Email, Password)

PurchaseDate	Cashier	ProductName	Price	UserName	Email	Password
Feb 11, 9:55	Mads	Tea	100	Martin	mhent	***
Feb 12, 10:03	Anna	Small	85	Martin	mhent	***
Feb 12, 10:05	Anna	Small	85	Omar	omsh	***
Feb 12, 10:06	Anna	Large	100	Omar	omsh	***

(Candidate) Key attributes:

PurchaseDate, ProductName, UserName, Email

Non-prime attributes:

Cashier, Price, Password

Full functional dependencies:

PurchaseDate → Cashier

ProductName → Price

UserName → (Email, Password)

Email → (UserName, Password)

Bringing into 2NF

Shift (Day, Cashier)

Day	Cashier
Feb 11	Mads
Feb 12	Anna

User (UserName, Email, Password)

UserName	Email	Password
Martin	mhent	***
Omar	omsh	***

Product (ProductName, Price)

ProductName	Price
Tea	100
Small	85
Large	100

Purchase (ProductDate, ProductName, UserName)

PurchaseDate	ProductName	UserName
Feb 11, 9:55	Tea	Martin
Feb 12, 10:03	Small	Martin
Feb 12, 10:05	Small	Omar
Feb 12, 10:06	Large	Omar

Third Normal Form (3NF) Definition

A relation R is in **third normal form (3NF)** if it satisfies 2NF and no non-prime attribute type of R is *transitively dependent* on the primary key.

A functional dependency $X \rightarrow Y$ is a **transitive dependency** if there is a set of attribute types Z that is neither a candidate key nor a subset of any key of R , and both $X \rightarrow Z$ and $Z \rightarrow Y$ hold.

Example user relation

User (UserName, Email, Password, University, UnivCity, UnivCountry)

UserName	Email	Password	University	UnivCity	UnivCountry
Martin	mhent	***	ITU	Copenhagen	Denmark
Omar	omsh	***	RU	Reykjavik	Iceland
Anna	anna	***	ITU	Copenhagen	Denmark

(Candidate) Key attributes: UserName, Email

Non-prime attributes: Password, University, UnivCity, UnivCountry

Full FDs: UserName → (Email, Password, University, UnivCity, UnivCountry)
Email → (UserName, Password, University, UnivCity, UnivCountry)
University → (UnivCity, UnivCountry)

Bringing into 3NF

Example user relation

User (UserName, Email, Password, University, UnivCity, UnivCountry)

UserName	Email	Password	University	UnivCity	UnivCountry
Martin	mhent	***	ITU	Copenhagen	Denmark
Omar	omsh	***	RU	Reykjavik	Iceland
Anna	anna	***	ITU	Copenhagen	Denmark

Transitive FDs: $\text{UserName} \rightarrow \text{University}$ and $\text{University} \rightarrow (\text{UnivCity}, \text{UnivCountry})$
 $\text{Email} \rightarrow \text{University}$ and $\text{University} \rightarrow (\text{UnivCity}, \text{UnivCountry})$

Data duplication. Risk of inconsistencies.

Bringing into 3NF

User (UserName, Email, Password, University, UnivCity, UnivCountry)

UserName	Email	Password	University	UnivCity	UnivCountry
Martin	mhent	***	ITU	Copenhagen	Denmark
Omar	omsh	***	RU	Reykjavik	Iceland
Anna	anna	***	ITU	Copenhagen	Denmark



User (UserName, Email, Password, University)

UserName	Email	Password	University
Martin	mhent	***	ITU
Omar	omsh	***	RU
Anna	anna	***	ITU

Uni (University, UnivCity, UnivCountry)

University	UnivCity	UnivCountry
ITU	Copenhagen	Denmark
RU	Reykjavik	Iceland

Equivalent 3NF Definition

A relation R is in 3NF if for each of its functional dependencies $X \rightarrow B$ with $X \subseteq R$ and $B \in R$, at least one of the following conditions hold:

- $B \in X$ (meaning $X \rightarrow B$ is a trivial FD)
- X is a superkey
- B is a prime attribute

Boyce-Codd Normal Form (BCNF)

A relation R is in BCNF if for each of its functional dependencies $X \rightarrow B$ with $X \subseteq R$ and $B \in R$, at least one of the following conditions hold:

- $B \in X$ (meaning $X \rightarrow B$ is a trivial FD)
- X is a superkey

(Removes last condition of alternative 3NF definition.)

Why Normal Forms?

Functional Dependencies

Normal Forms

- 1st Normal Form
- 2nd Normal Form
- 3rd Normal Form
- BC Normal Form

Normalization

Normalization

Normalization via Decomposition

- Look for functional dependencies
- Identify “bad” FD that breaks 3NF / BCNF
- If exists, decompose relation into two relations
- Repeat for new relations
- When done, database schema is normalized

→ Will result in BCNF or 3NF

- No way to go from 3NF to BCNF except by losing a dependency
- We don't decompose 3NF. 3NF good enough in practice!

Decomposition

Consider relation R and functional dependency $X \rightarrow Y$ that violates 3NF:

- Decompose R into R_1 and R_2 , where:
 $R_1 = R - Y$ (everything but Y)
 $R_2 = X \cup Y$ (the whole FD)

Example: Person(ID, Name, ZIP, City) and $\text{ZIP} \rightarrow \text{City}$

- $X = \text{ZIP}, Y = \text{City}$
- $R - Y = \text{Person}(\text{ID}, \text{Name}, \text{ZIP})$
- $X \cup Y = \text{ZIP}(\text{ZIP}, \text{City})$

Make sure to not lose a dependency. See example 5 below.

Example Decomposition 1

$R = \underline{AB}CD$

$AB \rightarrow C$

$C \rightarrow D$

$R_1 = \underline{ABC}$

$R_2 = \underline{CD}$

Both in BCNF.

$$X \rightarrow Y$$

$$R_1 = R - Y$$

$$R_2 = X \cup Y$$

Example Decomposition 2

$R = \underline{AB}CD$

$AB \rightarrow C$

$A \rightarrow D$

$R_1 = \underline{ABC}$

$R_2 = \underline{AD}$

Both in BCNF.

$$X \rightarrow Y$$

$$R_1 = R - Y$$

$$R_2 = X \cup Y$$

Example Decomposition 3

$$\begin{aligned} X &\rightarrow Y \\ R_1 &= R - Y \\ R_2 &= X \cup Y \end{aligned}$$

$$R = \underline{AB}CD$$

$$A \rightarrow C$$

$$A \rightarrow D$$

First, handle $A \rightarrow C$:

$$R_1 = \underline{AB}D$$

$$R_2 = \underline{AC}$$

Second, handle $A \rightarrow D$:

$$R_{11} = \underline{AB}$$

$$R_{12} = \underline{AD}$$

$$R_2 = \underline{AC}$$

Merge relations with same key:

$$S = \underline{AB}$$

$$T = \underline{ACD}$$

Both, S and T, in BCNF.

Example Decomposition 4

$$X \rightarrow Y$$

$$R_1 = R - Y$$

$$R_2 = X \cup Y$$

$R = \underline{A}BCD$

$D \rightarrow A$

R is in 3NF. No decomposition into BCNF.

Example Decomposition 5

$$X \rightarrow Y$$

$$R_1 = R - Y$$

$$R_2 = X \cup Y$$

$$R = \underline{ABCD}$$

$$A \rightarrow C$$

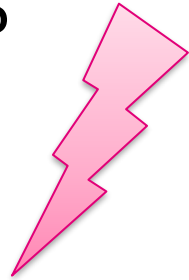
$$C \rightarrow D$$

Which FD to handle first and why?

Handle $A \rightarrow C$?

$$R_1 = \underline{ABD}$$

$$R_2 = \underline{AC}$$



Loses functional dependency $C \rightarrow D$!

Example Decomposition 5

$$X \rightarrow Y$$

$$R_1 = R - Y$$

$$R_2 = X \cup Y$$

$$R = \underline{A}BCD$$

$$A \rightarrow C$$

$$C \rightarrow D$$

Then handle $A \rightarrow C$:

$$R_{11} = \underline{AB}$$

$$R_{12} = \underline{AC}$$

$$R_2 = \underline{CD}$$

First handle $C \rightarrow D$:

$$R_1 = \underline{ABC}$$

$$R_2 = \underline{CD}$$

All three in BCNF.

Example Decomposition 6

$$X \rightarrow Y$$

$$R_1 = R - Y$$

$$R_2 = X \cup Y$$

Advises(Teacher, Student, Dept, Program, Course)

Teacher $\rightarrow$ Dept

Course $\rightarrow$ Program

Result:

Faculty(Teacher, Dept)

Catalog(Course, Program)

Advises(Teacher, Student, Course)

Takeaways

“Good” database design for your applications

- No duplicate data
- No anomalies and risk of inconsistencies
- No storage overhead

Chapter 6.2

Functional dependencies

Normal forms

- Foundation for schema refinement

Normalization process

- Decomposition of relations into 3NF / BCNF

